Chapter 20 exercises • Influential operating systems

OS20-E01 • Spool bottleneck

Three jobs J1, J2, J3 are all ready at time 0. Each needs 2 CPU time units, then submits output to a spool for 4 printer time units. One CPU and one printer each serve jobs nonpreemptively in J1, J2, J3 FIFO order; the devices can overlap, and queueing and transfers take no extra time. In the fully serial comparison, the next job starts only after the previous one is printed.

1. List all intervals.
2. Compare total/utilization with serial execution.
3. Distinguish spooling from multiprogramming.

OS20-E02 • THE and Mach boundaries

Compare THE software layers with a Mach 3 user server. For this question the client invokes a server in a different address space; IPC is successfully delivered and the server replies.

1. What responsibilities do both decompose?
2. Which specified path entails address-space crossing?
3. State conditions for layer-contract induction.

OS20-E03 • Atlas branches and faults

Frames have pairs (t, T): A(0,2), B(0,5), C(0,3); a separate set is A(2,4), B(1,6), C(0,8). Here t is the elapsed time since last use, and T is the preceding interval between uses. Use these rules in order: (1) if any $t > T + 1$, select an eligible frame; (2) otherwise, if some $t \neq 0$, select the largest $T - t$ among those frames; (3) if all $t = 0$, select the largest T. Break all ties in A, B, C order.

1. Select victims/branches.
2. For 512-word pages, VA 3597, page 7→frame 2, find PA.
3. Compare invariants with modern demand paging.

OS20-E04 • XDS relocation/sharing

Virtual memory is 16 Kiwords, physical memory 64 Kiwords, and each page 2 Kiwords; P page 3 maps to frame 11. Two processes each have eight resident pages. Their two corresponding code pages actually map to the same two physical frames; their six remaining pages each use distinct private frames. Exclude page-table and other overhead.

1. Count pages/frames and map VA 6161.
2. Count frames after sharing.
3. Does a page table prove demand paging?

OS20-E05 • Two-buffer conservation

Initially buffers b0 and b1 are free. C1 acquires b0 and C2 acquires b1 to enqueue their respective requests. S removes b0 and uses it to reply to C1; C1 has not consumed that reply and b1 remains queued. Each buffer occupies exactly one state at a time.

1. List free, queued, reply sets.
2. Can C3 claim b0 now? After C1 returns it?
3. Do atomic primitives prove deadlock freedom?

OS20-E06 • Exponential feedback

Let q_i = 2^i, starting at level 0, with a CPU burst of 11. Demote only when a full slice expires before completion; there is no competition, blocking, or promotion. Each dispatch incurs a fixed swap cost of 0.5, including the first dispatch and consecutive dispatches of the same job.

1. Give allowed/used/remaining per dispatch.
2. Compare total with RR q = 1.
3. Derive cumulative service and limits of the trace model.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

OS20-E07 • Multics segments/pages

Use the GE-645 segment/page address format: an 18-bit segment number, an 18-bit word offset and 1024-word pages. Segment 5 has a valid descriptor bounded to 3 pages; the segment permissions allow this read, and page 2 is present in frame 7. Translate offset 2500.

1. Split offset and calculate PA.
2. Why is a ring not a THE layer?
3. Why does resolved filename not authorize writing?

OS20-E08 • MFT/MVT and base-limit

Fixed partitions are four 64 KiB regions, with jobs 50, 50 and 70 KiB in that order; each job must fit in one region. The variable-partition holes are nonadjacent and have sizes 50, 30 and 40 KiB; no compaction or relocation is allowed. In a separate toy model, base = 256, limit = 768 and physical RAM = 1024.

1. Identify fragmentation.
2. Evaluate user addresses 0, 767, 768, −1.
3. Distinguish CP/CMS roles.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

OS20-E09 • Specified Mach 3 copying

C sends a 32 B request, and S replies with 4096 B. In each direction the data is copied twice through a kernel buffer; C and S have separate address spaces and no shared-page optimization. The successful call follows C user → C kernel → S kernel → S user, then follows the reverse path back to C user. Count user/kernel transitions as mode switches and the change between C kernel and S kernel as one address-space switch. Exclude additional interrupts, scheduling and system calls.

1. Count mode/address-space changes and copy bytes.
2. What does server-crash isolation guarantee or leave unresolved?
3. Why not apply it to Mach 2.x/every derivative?

OS20-E10 • Names versus mechanisms

A GUI has two windows; A loops in user code without yielding, B is ready. Elsewhere the interface completes a command name but the target lacks write authority. Command completion is not a successful write system call.

1. Compare cooperative/preemptive claims.
2. Does completion authorize writing?
3. Do high-level language/small kernel alone prove correctness?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 20. Atlas one-level store; THE system; Multics GE-645; Multics protection.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study